Online Appendix

"An Introduction to Double/Debiased Machine Learning"

S Monopsony in Online Markets

S.1 List of hand-engineered control variables

The hand-coded controls are taken from Dube et al. (2020), who, in turn, draw some of their variables from Difallah et al. (2015) and Gadiraju, Kawase, and Dietze (2014). Please see the Appendix of Dube et al. (2020) for more information.

Regular expressions. We apply regular expressions to each task's title, description, and keyword to create dummy variables:

> easy, transcr* (capturing, e.g., "transcription" or "transcribe"), writ* (capturing, e.g., "written", "write", or "writing"), audio, image|picture, video, bonus, copy, search, ident* (capturing, e.g., "identify"), text, date, fun, simpl*, summar*, only, improve, five|5, ?, and !

Task taxonomy. We include indicators for the following task categories suggested by Gadiraju, Kawase, and Dietze (2014):

- Information Finding
- Verification and Validation
- Interpretation and Analysis
- Content Creation
- Surveys
- Content Access

Additional hand-coded variables. Finally, again following Dube et al. (2020), we use the following variables as controls:

- `time_allotted`: The time a worker is given to complete a given task
- `first_hits`: The number of tasks initially posted to the marketplace
- `last_hits`: The number of tasks remaining to be completed in the group at the time it was last observed
- `max_hits`: The maximum number of tasks available observed for the group across all scrapes

- `avg_hitrate`: The average rate (per hour) at which tasks within the group were filled by workers
- `avg_hits_completed`: The average change in available tasks between subsequent observations of the group
- `med_hits_completed`: The median change in available tasks between subsequent observations of the group
- `min_hits_completed`: The minimum change in available tasks between subsequent observations of the group
- `max_hits_completed`: The maximum change in available tasks between subsequent observations of the group
- `num_zeros`: The number of observations for which the number of available tasks in the group was listed as 0
- `req_mean_reward`: The average reward over all tasks posted by the requester
- `req_mean_dur`: The average duration of all HIT groups posted by the requester
- `title_len`: The length of the HIT group's title
- `desc_len`: The length of the HIT group's description
- `keywords_len`: The sum of the lengths of the HIT group's keywords
- `num_keywords`: The number of keywords given for the HIT group
- `title_words`: The number of words in the HIT group's title
- `desc_words`: The number of words in the HIT group's description
- `minutes_title`: The number of minutes if a phrase including "X minutes" appears in the title
- `minutes_desc`: The number of minutes if a phrase including "X minutes" appears in the description
- `minutes_kw`: The number of minutes if a phrase including "X minutes" appears in the keyword list
- `qual_len`: The length of the string given in the HIT group's description which lists the qualifications
- `num_quals`: The number of qualifications required for the HIT group
- `custom_granted`: The number of custom qualifications required for the HIT group for which our "blank" account was qualified
- `any_loc`: A dummy variable representing whether or not the HIT group had a location restriction (e.g., US only)
- `us_only`: A dummy variable which is 1 if the HIT group is restricted to US workers, and 0 otherwise
- `appr_rate_gt`: The lower bound on approval rate required for workers to be eligible for the HIT, coded as -1 if no lower bound was enforced

- `appr_num_gt`: The lower bound on number of approvals required for workers to be eligible for the HIT, coded as -1 if no lower bound was enforced

S.2 Predictor transformations

Before estimation, we apply several transformations to the hand-coded control variables. First, we impute missing values for the HIT completion rate variables (`avg_hitrate`, `avg_hits_completed`, `med_hits_completed`, `min_hits_completed`, `max_hits_completed`) using their respective medians, as these variables have only a few missing observations.

We apply a $\log(x + 1)$ transformation to right-skewed continuous variables, including `time_allotted`, `first_hits`, `max_hits`, `avg_hitrate`, `avg_hits_completed`, `med_hits_completed`, `min_hits_completed`, `max_hits_completed`, `num_zeros`, `req_mean_reward`, `req_mean_dur`, `title_len`, `desc_len`, `num_keywords`, `title_words`, and `desc_words`. After the log transformation, we winsorize the HIT completion rate variables and `num_zeros` at the 99th percentile to limit the influence of extreme outliers.

Several variables with heavily concentrated or discrete distributions are replaced by grouped indicator variables. Specifically, `last_hits` is converted to a binary indicator for positive values, as it is near zero for most observations. `minutes_title` is replaced by three dummies indicating zero, 1–10, and more than 10 minutes; `minutes_kw` is reduced to a binary indicator for any mention of minutes. `qual_len` is grouped into zero, 1–100, and above 100; `num_quals` into zero, 1–3, and four or more. The approval rate threshold `appr_rate_gt` is split into three categories: not enforced (coded as -1), up to 90%, and above 90%. Similarly, `appr_num_gt` is grouped into not enforced, 0–99, 100–999, and 1,000 or more. The original ungrouped variables are dropped after the indicators are created.

Finally, the regular expression controls and Gadiraju task taxonomy indicators are binarized to 0/1 dummy variables.

S.3 Details on the fine-tuning process

For each cross-fitting fold and for each target variable (i.e., outcome and treatment), we fine-tune a pre-trained DeBERTa v3 (base) model using the concatenated task title and description. The text is tokenized up to a maximum length of 160 tokens. The model is then augmented with a linear regression head applied to the extracted "CLS" representation, and parameters are estimated by minimizing mean squared error. Fine-tuning is performed exclusively on the training folds to avoid data leakage, using the AdamW optimizer with a learning rate of 2×10^{-4} and three training epochs. When using Low-Rank Adaptation, we update only low-rank parameters (setting rank to $r = 16$)

in selected attention and feed-forward layers, while keeping the remaining pre-trained weights fixed. After fine-tuning, we discard the regression head and extract the fine-tuned text embeddings, which are then used as features (along with hand-coded controls) in the downstream nuisance-function estimation.

S.4 Alternative results

This section presents estimation results that complement Tables 5 and 6 in the main paper. First, we report results for $K = 5$ for each learner in Table S.1 and using meta learning approaches in Table S.2. Table S.1 shows that the `XGBoost` learners exhibit the best performance and are attributed the largest weight by stacking, similar to when $K = 3$. In Table S.2, we find an implied labor supply elasticity of around 0.05, which is close to our estimates of around 0.06 in the main text based on $K = 3$.

Table S.1: Complementary estimation results for the Monopsony application with $K = 5$

	Dependent variable: log duration					
Panel A.	(1)	(2)	(3)	(4)	(5)	(6)
log reward	−0.035	−0.030	−0.017	−0.236	−0.237	−0.240
	(0.061)	(0.057)	(0.062)	(0.126)	(0.127)	(0.129)
Out-of-sample R^2 Outcome	0.6753	0.6774	0.6526	0.1048	0.0996	0.1005
Out-of-sample R^2 Treatment	0.7347	0.7373	0.6907	0.4858	0.4833	0.4836
CVC p-value Outcome	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
CVC p-value Treatment	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Weights Outcome	0.0037	0.0021	-0.0000	-0.0000	0.0000	-0.0000
Weights Treatment	0.0805	0.2043	0.0000	-0.0000	-0.0000	0.0000
ML	OLS	CV-Lasso	CV-Ridge	RF 1	RF 2	RF 3
Panel B.	(7)	(8)	(9)	(10)	(11)	(12)
log reward	−0.039	−0.058*	−0.050	−0.061	−0.073	−0.035
	(0.035)	(0.026)	(0.029)	(0.039)	(0.058)	(0.031)
Out-of-sample R^2 Outcome	0.8451	0.8617	0.8632	0.8406	0.8206	0.8165
Out-of-sample R^2 Treatment	0.7308	0.7471	0.7387	0.7355	0.7164	0.7012
CVC p-value Outcome	0.0000	0.3048	0.8056	0.0000	0.0000	0.0000
CVC p-value Treatment	0.1996	0.8968	0.0000	0.0000	0.0000	0.0000
Weights Outcome	0.1922	0.2415	0.2991	0.1210	0.0667	0.0736
Weights Treatment	0.1577	0.3047	0.0858	0.0622	0.0587	0.0460
ML	XGB 1	XGB 2	XGB 3	NN 1	NN 2	NN 3

Notes: The table reports DML estimates based on 12 different nuisance estimators. The controls are the hand-coded controls of Dube et al. (2020) and DeBERTa embeddings, fine-tuned using LoRA. We use $K = 5$ cross-fitting folds, implemented by randomly assigning recruiters to folds, and report median aggregated estimates obtained using $S = 5$ cross-fitting repetitions. The number of observations is $N = 258\,352$. The diagnostics reported are the cross-fitted R^2, the CVC p-value, and the short-stacking weights, each averaged across cross-fitting repetitions. We consider the following nuisance estimators: OLS; CV-lasso (lasso with tuning parameter selected by cross-validation); CV-ridge (ridge with tuning parameter selected by cross-validation); three types of random forest labeled RF 1 (600 trees), RF 2 (600 trees, minimum terminal node size of 30) and RF 3 (600 trees, minimum terminal node size of 100); three types of `XGBoost` labeled XGB 1 (800 trees, early stopping after 10 rounds, 10% evaluation set), XGB 2 (800 trees, minimum node size of 500) and XGB 3 (800 trees, minimum node size of 2000); three neural networks denoted NN 1 (3 hidden layers of size 10, 500 epochs), NN 2 (3 hidden layers of size 30) and NN 3 (5 hidden layers of size 50). DML estimation uses the `R` package `ddml` (Wiemann et al., 2023). The nuisance estimators were implemented with `glmnet` (Friedman, Hastie, and Tibshirani, 2010), `ranger` (Wright and Ziegler, 2017), `XGBoost` (Chen and Guestrin, 2016), and `keras` (Kalinowski, Allaire, and Chollet, 2025). Standard errors are clustered at the recruiter level. * p <0.05, ** p <0.01, *** p <0.001

Table S.2: Complementary estimation results for the Monopsony application: Meta learning approaches with $K = 5$

	Dependent variable: log duration		
	Stacking	Single-best	CVC
log reward	−0.053*	−0.048	−0.047
	(0.027)	(0.031)	(0.030)
R^2 Outcome	0.8754	0.8634	0.8649
R^2 Treatment	0.7632	0.7472	0.7493

Notes: The table reports DML estimates when several learners are combined by constrained least squares (imposing non-negative weights that sum to one), by selecting the single best learner for each equation, or by assigning equal weights to all learners for which the CVC p-values are larger than 0.2. We use $K = 5$ cross-fitting folds, implemented by randomly assigning recruiters to folds, and report median aggregated estimates obtained using $S = 5$ cross-fitting repetitions. Standard errors are clustered at the recruiter level. * p <0.05, ** p <0.01, *** p <0.001

Second, Table S.3 compares results under alternative approaches to fold construction. Panel A reproduces the meta learning results from the main paper (Table 6), where sample-splitting is performed at the recruiter level (ensuring that all recruiter observations are assigned to the same fold), and standard errors are clustered by recruiter. Panel B reports estimates obtained when folds are constructed at the observation level rather than the recruiter level, thereby ignoring the dependence structure in the data. The standard errors are again clustered at the recruiter level.

We make two observations: First, the out-of-sample predictive performance is lower by around 2–3 percentage points, suggesting that the learners partially rely on within-cluster similarities. Second, standard errors are roughly 3–4 times smaller when within-recruiter dependence is ignored in the cross-fitting step. Together, the comparisons underscore the importance of accounting for dependence structures not only in variance estimation but also in the construction of cross-fitting folds.

Table S.3: Complementary estimation results for the Monopsony application: Meta learning approaches with and without fold split by cluster, $K = 3$

	Dependent variable: log duration		
	Stacking	Single-best	CVC
Panel A. Cross-fitting split by recruiter			
log reward	−0.064***	−0.061***	−0.063***
	(0.019)	(0.016)	(0.017)
R^2 Outcome	0.8748	0.8625	0.8640
R^2 Treatment	0.7630	0.7476	0.7534
Panel B. Cross-fitting split by observation			
log reward	−0.045***	−0.039***	−0.042***
	(0.005)	(0.005)	(0.005)
R^2 Outcome	0.9027	0.8927	0.8967
R^2 Treatment	0.8900	0.8819	0.8869

Notes: The table compares meta-learning results when cross-fitting folds are split by recruiter (Panel A) or by observation (Panel B). Each panel reports DML estimates when several learners are combined by constrained least squares (imposing non-negative weights that sum to one), by selecting the single best learner for each equation, or by assigning equal weights to all learners for which the CVC p-values are larger than 0.2. We use $K = 3$ cross-fitting folds, implemented by randomly assigning recruiters to folds, and report median aggregated estimates obtained using $S = 5$ cross-fitting repetitions. Standard errors are clustered at the recruiter level. * p <0.05, ** p <0.01, *** p <0.001

References

Chen, Tianqi and Carlos Guestrin (2016). "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. San Francisco, USA: ACM, pp. 785–794.

Difallah, Djellel, Michele Catasta, Gianluca Demartini, Panagiotis Ipeirotis, and Philippe Cudre-Mauroux (2015). "The Dynamics of Micro-Task Crowdsourcing: The Case of Amazon MTurk". In: pp. 238–247.

Dube, Arindrajit, Jeff Jacobs, Suresh Naidu, and Siddharth Suri (2020). "Monopsony in Online Labor Markets". *American Economic Review: Insights* 2.1, pp. 33–46.

Friedman, Jerome, Trevor Hastie, and Robert Tibshirani (2010). "Regularization Paths for Generalized Linear Models via Coordinate Descent". *Journal of Statistical Software* 33.1, pp. 1–22.

Gadiraju, Ujwal, Ricardo Kawase, and Stefan Dietze (2014). "A taxonomy of microtasks on the web". In: *Proceedings of the 25th ACM Conference on Hypertext and Social Media*. HT '14. Santiago, Chile: Association for Computing Machinery, 218–223. URL: `https://doi.org/10.1145/2631775.2631819`.

Kalinowski, Tomasz, JJ Allaire, and François Chollet (2025). *keras3: R Interface to 'Keras'*.

Wiemann, Thomas, Achim Ahrens, Christian B Hansen, and Mark E Schaffer (2023). "`ddml`: Double/Debiased Machine Learning in R".

Wright, Marvin N. and Andreas Ziegler (2017). "`ranger`: A Fast Implementation of Random Forests for High Dimensional Data in C++ and R". *Journal of Statistical Software* 77.1, pp. 1–17.